# Especificación de Requisitos de Software

## AI Software Engineering System

Documento SRS 1.0 · Borrador para revisión · 21 de octubre de 2026

Este documento especifica el comportamiento requerido de AI Software Engineering System y delimita el producto local previsto para v0.1.0. Utiliza la organización de IEEE 830 para facilitar la revisión académica y técnica. No acredita una certificación de conformidad ni declara terminada la versión de producto.

La base disponible es un candidato local funcional. El laboratorio de modelos está parcialmente preparado; integración validada de modelos reales, voz real, ejecutable e instalador siguen pendientes. Esta distinción se mantiene en cada requisito para evitar presentar objetivos como capacidades entregadas.

### Control del documento

| Campo | Valor |
|---|---|
| Identificador | SRS AI SWE 001 |
| Versión documental | 1.0 |
| Producto objetivo | AI Software Engineering System v0.1.0 local |
| Equipo | Equipo 4 |
| Destinatarios | Responsable del proyecto, equipo de desarrollo y revisión académica |
| Estado de aprobación | Pendiente de revisión del responsable del proyecto |
| Base de referencia | delivery-rc1 en 2d35a59; checkpoint de productización en 7419818 |
| Idioma | Español |

### Historial

| Versión | Fecha | Cambio |
|---|---|---|
| 1.0 | 2026-10-21 | Primera especificación consolidada en estructura IEEE 830 |

### Contenido

1. Introducción: propósito, alcance, definiciones y referencias. 2. Descripción general: contexto, funciones, usuarios y restricciones. 3. Requisitos específicos: interfaces, funciones, datos y atributos de calidad. 4. Apéndices: casos de uso, estados, correspondencia y decisiones pendientes.

# 1 Introducción

## 1.1 Propósito

Definir los requisitos del sistema para que sus responsables puedan revisar qué debe hacer, qué límites debe respetar y qué condiciones permiten aceptar sus entregables. La especificación sirve como referencia para implementación, revisión y control de cambios; no sustituye los contratos ejecutables del código.

Los identificadores SRS F, SRS N y SRS I son propios de este documento. No renumeran los RF y RNF de la arquitectura previa. La correspondencia de alcance se recoge en el apéndice C.

## 1.2 Alcance del producto

El sistema recibirá una instrucción de ingeniería de software escrita o derivada de una transcripción confirmada. Interpretará la petición, solicitará validación humana, coordinará agentes especializados y propondrá cambios sobre un repositorio Git autorizado. Los cambios se prepararán en un worktree separado y solo se promoverán al repositorio original tras aprobación humana explícita.

El producto v0.1.0 está orientado a uso local, una persona por instalación y servidor ligado a 127.0.0.1. La interfaz principal es un navegador. Un producto Windows instalable es objetivo de productización, no un entregable ya disponible.

Quedan fuera de v0.1.0: multiusuario, OAuth, despliegue cloud del producto, alta disponibilidad distribuida, MCP, Graphify, Knowledge Graph, TTS, integración completa de Langfuse, PostgreSQL, Kubernetes, firma de código, TLS de producción, aislamiento completo de procesos y el ticket SSE de un solo uso TD 02. La cancelación activa y el borrado o retención automatizados tampoco forman parte del compromiso de esta versión mientras no exista implementación aprobada.

## 1.3 Definiciones y abreviaturas

| Término | Significado en este proyecto |
|---|---|
| SRS o ERS | Especificación de Requisitos de Software |
| Run | Ejecución identificable de una instrucción |
| HITL | Intervención humana obligatoria en un punto de decisión |
| Worktree | Árbol Git separado donde se aplican los cambios propuestos |
| Promoción | Aplicación del cambio aprobado al repositorio autorizado |
| ModelGateway | Frontera común de acceso a proveedores de modelos |
| Adapter | Implementación del contrato de comunicación con un proveedor |
| STT | Conversión de audio a texto |
| SSE | Canal HTTP de eventos del servidor hacia el navegador |
| Budget | Límites por ejecución de recursos y ciclos permitidos |
| Checkpoint | Estado persistido que permite reanudar un grafo interrumpido |
| Model Lab | Herramientas externas para descubrir y evaluar modelos |

### 1.4 Referencias

- IEEE 830-1998, Recommended Practice for Software Requirements Specifications:

https://standards.ieee.org/ieee/830/1222/

- ISO/IEC/IEEE 29148:2018, Requirements engineering:

https://standards.ieee.org/ieee/29148/6937/

- docs/01-arquitectura.md: arquitectura, decisiones y requisitos previos.
- docs/03-flujo-e-implementacion.md: flujo y responsabilidades del sistema.
- docs/07-contrato-frontend-backend.md: coordinación entre UI y API.
- docs/RELEASE-CANDIDATE-PHASE-C.md: alcance del candidato local.
- docs/PRODUCTIZATION-PROGRESS.md y docs/MODEL-LAB-REPORT.md: avance parcial.
- src/ai_swe/contracts/: contratos Pydantic; src/ai_swe/api/app.py: API efectiva.

IEEE 830 es una referencia histórica reemplazada por ISO/IEC/IEEE 29148. Se usa su estructura por solicitud del proyecto, con redacción original y sin reproducir el texto de la norma. Las fuentes internas contienen información de fechas distintas: una afirmación histórica no modifica el comportamiento actual.

### 1.5 Organización y convenciones

El capítulo 2 explica el contexto. El capítulo 3 contiene requisitos identificados y criterios de aceptación; los apéndices facilitan su lectura y correspondencia.

Cada requisito expresa una obligación mediante “deberá”. Prioridad A significa esencial para el alcance local; prioridad B, complementaria. El estado I indica implementación disponible; P, implementación o validación parcial; O, objetivo de productización pendiente. Estos estados describen la referencia consultada, no una aprobación formal del documento ni una nueva certificación de ejecución.

## 2 Descripción general

### 2.1 Perspectiva del producto

La aplicación combina frontend React y TypeScript, API FastAPI, coordinación LangGraph, persistencia SQLite y herramientas locales controladas por políticas. RunEngine y RunWorker separan la aceptación HTTP del trabajo del grafo. La implementación actual utiliza trabajadores en hilos; no se afirma ejecución en un servicio separado ni ausencia de estado operativo en memoria.

Los agentes no implementan proveedores directamente: usan ModelGateway. Existen un adapter determinista para operación de desarrollo y un adapter HTTP compatible con OpenAI. Su existencia no acredita validación real con todos los proveedores.

El Model Lab es externo. El notebook prepara experimentos y recomendaciones; la aplicación no depende de que Jupyter o una GPU cloud estén encendidos. En el checkpoint actual solo hay descubrimiento público y preparación offline.

### 2.2 Funciones principales

- Registrar repositorios Git y consultar proyectos.
- Capturar una instrucción, interpretarla y confirmar sus requisitos.
- Descomponer el trabajo y coordinar investigación, código y revisión.

- Aplicar propuestas en worktrees y conservar evidencia de su ejecución.
- Solicitar una decisión antes de promover cambios al repositorio original.
- Consultar estados, eventos, diferencias, revisión e informe de un run.
- Transcribir audio de manera opcional, siempre con revisión humana del texto.
- Controlar alcance, permisos, recursos y exposición accidental de secretos.

## 2.3 Clases de usuarios

El usuario principal es una persona con conocimientos básicos de repositorios, cambios de código y revisión de resultados. Debe poder comprender las decisiones de los gates sin conocer el funcionamiento interno de LangGraph.

Si se habilita autenticación por API key, reader podrá consultar, operator podrá iniciar y decidir runs y admin podrá registrar proyectos. Son permisos de claves locales, no cuentas multiusuario ni un sistema de identidad organizacional. Sin claves configuradas no se aplicará esa separación de roles.

## 2.4 Entorno de operación

La ejecución desde fuentes requiere Python compatible con el manifiesto, las dependencias declaradas y Git. La construcción del frontend requiere Node y npm; servir el frontend ya compilado no requiere ejecutar Vite en desarrollo.

El producto empaquetado tendrá Windows como plataforma objetivo y deberá operar sin Python o Node instalados por separado. Esa independencia todavía está pendiente de implementación y validación. No se establecen mínimos universales de RAM o VRAM: dependen del modelo elegido y deberán documentarse antes de release.

## 2.5 Restricciones

El servicio local deberá mantenerse en loopback. No se ejecutarán órdenes arbitrarias sugeridas por un modelo. Las credenciales serán externas al código y a los artefactos distribuibles. Los agentes solo podrán usar herramientas y rutas permitidas. El worktree confina escrituras, pero no garantiza impedir acceso a red, consumo de recursos o efectos de procesos ejecutados por código no confiable.

La interfaz no deberá ofrecer como funcionales operaciones sin implementación. El endpoint de cancelación que responde 501 no constituye cancelación operativa. El modo de desarrollo determinista no deberá presentarse como inferencia real.

## 2.6 Suposiciones y dependencias

Se supone que el usuario tiene permisos de lectura y escritura sobre el repositorio registrado y espacio suficiente para worktrees, base de datos y artefactos. Un repositorio inaccesible o inválido deberá producir un error legible.

La inferencia externa depende de credenciales, disponibilidad y políticas del proveedor. La transcripción local depende del adapter, biblioteca y modelo instalados. La presencia del adapter no implica que el modelo esté cargado o que CUDA funcione. Los componentes opcionales no deberán impedir el uso disponible.

### 2.7 Evolución prevista

La secuencia aprobada es: laboratorio de modelos, integración progresiva de proveedores reales, STT real, configuración de producto, launcher, empaquetado, instalador y cierre de v0.1.0. El trabajo de productización permanece pausado por solicitud del usuario; este documento no lo reanuda ni autoriza consumo externo.

## 3 Requisitos específicos

### 3.1 Interfaces externas

#### SRS I01 Interfaz de usuario

Prioridad A · Estado I. El sistema deberá proporcionar vistas de proyectos, ejecuciones, detalle, flujo, seguridad y sesión, además de una vista general con captura de instrucciones. Deberá mostrar carga, errores y decisiones pendientes. Aceptación: cada acción visible conduce a una operación existente y muestra su resultado; ninguna tarjeta declara progreso o capacidades inventadas.

#### SRS I02 Interfaz HTTP

Prioridad A · Estado I. La API deberá intercambiar JSON validado y utilizar códigos HTTP coherentes con aceptación, validación, autorización y disponibilidad. Aceptación: POST /runs devuelve 202 con el identificador sin esperar a que termine el grafo; peticiones inválidas no se convierten en acciones aprobadas.

| Operación | Interfaz principal |
|---|---|
| Salud del proceso | GET /health |
| Proyectos | GET y POST /projects; GET /projects/{project_id} |
| Runs | POST y GET /runs; GET /runs/{run_id} |
| Progreso | GET /runs/{run_id}/events mediante SSE |
| Confirmación de requisitos | POST /runs/{run_id}/confirm-requirements |
| Decisión sobre el cambio | POST /runs/{run_id}/approve |
| Transcripción opcional | POST /transcriptions mediante carga de audio |
| Frontend compilado | /app |

La respuesta 501 de /runs/{run_id}/cancel deberá mantenerse identificada como función no implementada. Esta SRS no presupone un endpoint /trace inexistente.

#### SRS I03 Progreso SSE

Prioridad A · Estado I. El navegador deberá consumir eventos identificados por run y reanudar su lectura sin confundir ejecuciones. Aceptación: atravesar un gate no cierra definitivamente el progreso; seleccionar otro run no muestra datos tardíos del anterior. Un canal cerrado por estado terminal se distingue de una espera.

#### SRS I04 Sistemas y dispositivos externos

Prioridad A · Estado P. Git será la interfaz del repositorio y los proveedores usarán el adapter configurado. El micrófono se utilizará únicamente con permiso. Aceptación: un dispositivo o proveedor ausente produce un mensaje explícito y no se sustituye silenciosamente por contenido fabricado.

### 3.2 Requisitos funcionales

#### SRS F01 Captura de la instrucción

Prioridad A · Estado I. El sistema deberá aceptar instrucciones de texto no vacías y permitir su edición antes del envío. Aceptación: únicamente una acción explícita del usuario crea el run; un borrador o cambio de vista no lo crea.

#### SRS F02 Transcripción revisable

Prioridad B · Estado P. El sistema deberá convertir audio admitido en texto editable y metadatos de transcripción cuando el servicio STT esté disponible. Aceptación: transcribir no crea un run; el usuario puede corregir, confirmar o descartar el texto. La calidad del micrófono y STT reales sigue pendiente.

#### SRS F03 Procedencia de voz

Prioridad A · Estado I. Una instrucción de origen voice deberá incluir confirmación humana y metadatos de transcripción compatibles con el contrato. Aceptación: audio crudo o texto de voz no confirmado no llegan al grafo como una orden autorizada.

#### SRS F04 Registro de proyectos

Prioridad A · Estado I. El sistema deberá registrar un nombre y una ruta Git válida, asignar identificador y permitir consulta. Aceptación: rutas inválidas o fuera de la lista autorizada configurada se rechazan antes de ejecutar trabajo.

#### SRS F05 Alcance por ejecución

Prioridad A · Estado I. El sistema deberá conservar las rutas permitidas y protegidas de cada run y aplicarlas a las operaciones de escritura y promoción. Aceptación: una ruta denegada no se escribe por cambiar su representación o introducir un escape.

#### SRS F06 Creación y consulta de runs

Prioridad A · Estado I. Cada ejecución deberá disponer de run_id, instrucción, opciones, estado y fechas. Aceptación: el cliente puede consultar el run creado y obtener su situación, sin depender de una petición HTTP que espere todo el trabajo.

#### SRS F07 Interpretación estructurada

Prioridad A · Estado I. La interpretación deberá producir TaskInterpretation con intención, alcance, restricciones y explicación. Aceptación: una salida que no valide no se incorpora como interpretación válida; cualquier fallback queda señalado.

#### SRS F08 Confirmación humana de requisitos

Prioridad A · Estado I. El sistema deberá detener el flujo antes de planificar cuando corresponda confirmar la interpretación. Aceptación: confirmar reanuda; corregir conserva las modificaciones admitidas; rechazar no continúa como aceptación.

**SRS F09 Planificación**

Prioridad A · Estado I. El Supervisor deberá generar un RunPlan con subtareas, agente, herramientas, dependencias y criterio de término. Aceptación: no se admiten identificadores duplicados, ciclos o dependencias inexistentes en el plan validado.

**SRS F10 Coordinación de especialistas**

Prioridad A · Estado I. El grafo deberá coordinar Researcher, Coder y Reviewer según el plan y la ruta de la instrucción. Aceptación: una pregunta o inspección no se presenta obligatoriamente como una modificación de código completada.

**SRS F11 Investigación determinista**

Prioridad A · Estado I. Researcher deberá recopilar información mediante las herramientas locales permitidas y conservar hallazgos. Aceptación: no requiere convertirse en un agente LLM ni depende de MCP o Knowledge Graph.

**SRS F12 Propuesta de código**

Prioridad A · Estado I. Coder deberá producir CodeProposal con operaciones y contenido de archivos. El sistema derivará diff y metadatos. Aceptación: no se confía en hashes inventados por el modelo como prueba de integridad del cambio.

**SRS F13 Aplicación separada del repositorio original**

Prioridad A · Estado I. El sistema deberá aplicar el cambio en un worktree Git separado. Aceptación: antes de la aprobación, el contenido y el estado Git del repositorio original no cambian como consecuencia del parche propuesto.

**SRS F14 Ejecución de comprobaciones permitidas**

Prioridad A · Estado I. El sistema deberá ejecutar exclusivamente targets configurados de la tabla de comandos permitidos y conservar su salida, código de retorno y duración. Aceptación: no convierte texto arbitrario del modelo en shell.

**SRS F15 Revisión estructurada**

Prioridad A · Estado I. Reviewer deberá emitir ReviewVerdict con accept, request_changes o escalate sobre el cambio y su evidencia. Aceptación: solicitar correcciones exige indicaciones concretas y no se presenta como aceptación.

**SRS F16 Correcciones acotadas**

Prioridad A · Estado I. El sistema deberá permitir volver al Coder por revisión o solicitud humana dentro del límite de ciclos. Aceptación: al agotar el límite no continúa indefinidamente y expone la necesidad de intervención.

**SRS F17 Decisión humana sobre el cambio**

Prioridad A · Estado I. Antes de promover, el usuario deberá poder consultar diff, evidencia y revisión, y decidir approved, modified o rejected. Aceptación: no hay aprobación predeterminada; modified conserva el feedback y no promueve todavía.

**SRS F18 Promoción íntegra**

Prioridad A · Estado I. Solo una aprobación explícita deberá permitir aplicar al repositorio autorizado el contenido revisado. Aceptación: se verifican destino, alcance y diff; después de aplicar, los bytes coinciden con el worktree. No se crea un commit ni se hace push de forma implícita.

### SRS F19 Error y rechazo de promoción

Prioridad A · Estado I. Un rechazo o fallo de promoción no deberá anunciarse como cambio aplicado con éxito. Aceptación: rechazado conserva el árbol original; un error de promoción queda declarado y la interfaz no muestra promoción completada.

### SRS F20 Informe y consulta histórica

Prioridad A · Estado I. El sistema deberá conservar plan, diff, evidencia, aprobaciones y el informe final que corresponda al desenlace. Aceptación: se distinguen cambios aplicados, comprobaciones realizadas, revisión y pendientes; un run escalado puede carecer de informe de éxito.

### SRS F21 Recursos y costes

Prioridad A · Estado P. El sistema deberá aplicar los límites efectivos de tokens, llamadas y ciclos disponibles y registrar usage fiable. Aceptación: un dato de coste desconocido no se convierte en coste cero confirmado. La medición continua del tiempo y disponibilidad global de coste tienen limitaciones documentadas.

### SRS F22 Permisos y sesión

Prioridad A · Estado I. Con autenticación habilitada, la API deberá autorizar cada operación según la jerarquía reader, operator y admin. Aceptación: 401 invalida la sesión de UI; 403 informa falta de permiso sin confundirla con una clave inválida.

### SRS F23 Laboratorio externo

Prioridad B · Estado P. El laboratorio deberá descubrir modelos dinámicamente, evaluarlos por rol y producir resultados sin secretos. Aceptación: catálogo sin inferencia se marca DISCOVERY_ONLY y no produce recomendaciones de calidad ficticias.

### SRS F24 Routing real progresivo

Prioridad A · Estado O. El producto deberá permitir configuración tipada por rol basada en evidencia del laboratorio. Aceptación: Analyst, Supervisor, Coder y Reviewer pueden integrarse progresivamente; la ruta efectiva y el fallback son explícitos. Los adapters fake permanecen disponibles para desarrollo offline.

### SRS F25 Consentimiento de consumo externo

Prioridad A · Estado P. Antes de inferencia externa deberá comprobarse credencial y autorización de consumo cuando pueda generar cargo. Aceptación: ningún token, precio de catálogo o crédito supuesto constituye por sí mismo permiso de gasto.

### SRS F26 Datos de producto instalado

Prioridad A · Estado O. La instalación Windows deberá separar binarios, datos, configuración, logs y caché. Aceptación: SQLite no se almacena en Program Files; los datos de usuario se ubican bajo LOCALAPPDATA/AISWESystem o configuración equivalente.

### SRS F27 Arranque mediante launcher

Prioridad A · Estado O. Un launcher deberá seleccionar un puerto loopback disponible, iniciar el servidor, esperar su salud y abrir la URL efectiva. Aceptación: un puerto ocupado no abre el navegador en otro servicio; los errores de configuración, base de datos o assets son legibles y permiten diagnóstico.

#### SRS F28 Paquete e instalador Windows

Prioridad A · Estado O. El producto deberá distribuirse inicialmente como onedir, con frontend compilado y sin secretos ni artefactos de desarrollo. Aceptación: puede instalarse, abrirse, cerrarse y reabrirse conservando datos sin depender de Python, Node, npm, entorno virtual o editor externos. No se añaden servicios Windows.

### 3.3 Requisitos de rendimiento

#### SRS N01 Respuesta y progreso

Prioridad A · Estado I. La API deberá aceptar el run separadamente de su ejecución y la UI deberá mostrar carga y progreso incremental. Aceptación: la navegación no depende de mantener una petición de creación abierta durante todo el grafo. No se fija un SLA numérico sin hardware y carga de referencia acordados.

#### SRS N02 Límites de entrada y consulta

Prioridad A · Estado I. El sistema deberá aplicar topes configurables de audio, paginación y conexiones SSE, y rate limiting cuando se habilite. Aceptación: superar un tope produce un error explícito y no un consumo ilimitado silencioso.

#### SRS N03 Finalización acotada

Prioridad A · Estado P. Los proveedores y comandos deberán tener timeout y los ciclos de revisión un límite. Aceptación: un timeout o agotamiento genera un resultado declarado; no se promete contención completa de subprocesos o recursos.

### 3.4 Requisitos de datos

#### SRS N04 Persistencia y reanudación

Prioridad A · Estado I. SQLite deberá conservar proyectos, ejecuciones y checkpoints necesarios para reanudar esperas humanas. Aceptación: reanudar una puerta inexistente se rechaza; un run activo huérfano se identifica al reiniciar. No se promete recuperación automática de cualquier proceso interrumpido.

#### SRS N05 Trazabilidad de decisiones

Prioridad A · Estado I. Cada aprobación deberá asociarse al run, acción, actor declarado, fecha y huella del contenido. Aceptación: puede distinguirse la última decisión de decisiones anteriores y recuperar qué contenido se aprobó.

#### SRS N06 Integridad de contratos

Prioridad A · Estado I. Las entradas y salidas de agentes deberán validarse con contratos Pydantic. Los contratos exportados al frontend deberán permanecer sincronizados. Aceptación: un objeto mal formado no se usa como salida válida; no se afirma validación exhaustiva de JSON Schema en el navegador.

### 3.5 Restricciones de diseño

#### SRS N07 Operación local

Prioridad A · Estado I. El arranque local deberá enlazar 127.0.0.1, no todas las interfaces. Aceptación: la configuración ordinaria del producto no amplía la exposición de red. La administración remota no forma parte del alcance.

#### SRS N08 Independencia del laboratorio

Prioridad A · Estado P. El runtime no deberá importar el notebook ni requerir Jupyter o GPU cloud para iniciar. Aceptación: el producto local puede funcionar con las capacidades disponibles aunque el entorno experimental esté apagado.

#### SRS N09 Sustitución de proveedor

Prioridad A · Estado I. La comunicación con modelos deberá pasar por ModelGateway y adapters. Aceptación: las decisiones del grafo no dependen de un SDK HF dentro de cada agente. La compatibilidad con un proveedor concreto debe validarse aparte.

### 3.6 Atributos de calidad

#### SRS N10 Protección de secretos

Prioridad A · Estado P. El sistema deberá usar secretos externos y redactarlos antes de su exposición en registros y artefactos. Aceptación: no se incluyen en Git, notebook, resultados del laboratorio ni instalador. La API key de UI vive en memoria; el parámetro SSE actual sigue siendo una limitación TD 02 reconocida.

#### SRS N11 Mínimo privilegio y fronteras

Prioridad A · Estado I. El sistema deberá denegar herramientas, comandos y rutas no autorizados, y tratar el contenido del repositorio como datos no confiables. Aceptación: la autorización no depende únicamente de que el modelo obedezca un prompt. No se declara implementado bloqueo completo de red del runner.

#### SRS N12 Información comprensible

Prioridad A · Estado I. La UI deberá explicar estados, gates, errores y operaciones no disponibles sin atribuir éxito por la simple aprobación. Aceptación: no muestra porcentajes artificiales, costes confirmados inexistentes ni herramientas futuras como si estuvieran activas.

#### SRS N13 Accesibilidad y adaptación

Prioridad A · Estado P. Los controles principales deberán permitir teclado, etiquetas comprensibles, foco visible y composición adaptable a móvil y escritorio. Aceptación: las acciones no dependen solo del color o del clic en una fila. La evaluación completa con lector de pantalla permanece pendiente.

#### SRS N14 Manejo de fallos del proveedor

Prioridad A · Estado P. El acceso al modelo deberá distinguir errores temporales de autenticación, petición permanente y agotamiento de recursos. Aceptación: los reintentos son acotados y no se repite indefinidamente una credencial rechazada; un fallback utilizado debe declararse como tal.

### 3.7 Otros requisitos

El producto deberá presentar la licencia y restricciones de sus dependencias y modelos cuando corresponda, sin aceptar por el usuario términos que exijan su consentimiento. No se incorporan obligaciones de una legislación concreta ni se declara cumplimiento normativo de privacidad sin análisis adicional.

La documentación de instalación, primer uso, proveedores, voz y limitaciones deberá acompañar el producto antes de declarar v0.1.0 completo. La estructura documental solicitada excluye carpetas de pruebas y métricas; ello no elimina los controles ni las comprobaciones internas del software existente.

# 4 Apéndices

## Apéndice A Casos de uso principales

| Caso | Actor y precondición | Resultado requerido |
|---|---|---|
| CU 01 Registrar proyecto | Admin si auth está activa; ruta Git accesible | Proyecto persistido o error legible |
| CU 02 Iniciar trabajo | Operator o modo local; texto confirmado | Run aceptado con identificador y estado consultable |
| CU 03 Confirmar requisitos | Run esperando interpretación | Confirmación o corrección reanuda; rechazo no autoriza planificación |
| CU 04 Solicitar correcciones | Run esperando aprobación del cambio | Feedback conservado y retorno al ciclo sin promover |
| CU 05 Aprobar cambio | Diff revisado y decisión explícita | Promoción íntegra o error declarado; nunca éxito ficticio |
| CU 06 Rechazar cambio | Run esperando decisión | Finalización sin aplicar al repositorio original |
| CU 07 Dictar instrucción | STT y permiso de audio disponibles | Transcripción editable; run solo tras confirmación y ejecución explícita |
| CU 08 Consultar historial | Reader o modo local | Estado, plan, cambios y evidencia disponibles |

## Apéndice B Estados de una ejecución

| Estado | Significado |
|---|---|
| planned | Run aceptado, aún no iniciado |
| running | Trabajo en ejecución |
| awaiting_human_requirements_confirmation | Espera legítima de validación de requisitos |
| awaiting_approval | Espera legítima de decisión sobre el cambio |
| completed | Finalización correcta de la ruta ejecutada; puede no incluir cambios |
| needs_human | Escalado terminal con intervención necesaria |
| failed | Fallo terminal declarado |
| cancelled | Rechazo/cierre cancelado; no acredita cancelación activa por API |

No todas las rutas recorren todos los estados. Los estados de espera no son terminales. Tras una aprobación se comprueba el resultado de promoción antes de anunciar cambios aplicados. El éxito de una consulta no implica que hubo parche.

## Apéndice C Correspondencia con el proyecto

| Tema | Requisitos de esta SRS | Fuente técnica |
|---|---|---|
| Entrada y voz | F01 a F03; I04 | contracts/state.py; stt/base.py; componentes de voz |
| API y proyectos | F04 a F06; I01 a I03 | api/app.py; frontend/src/lib/api.ts |
| Requisitos y plan | F07 a F10 | contracts/tasks.py; graph/nodes.py; graph/workflow.py |
| Trabajo de especialistas | F11 a F16 | graph/nodes.py; tools/patcher.py; tools/runner.py |
| Gates y promoción | F17 a F20; N04 y N05 | graph/engine.py; tools/promoter.py; state/store.py |
| Modelos y recursos | F21; F23 a F25; N08, N09 y N14 | models/; tools/model_lab*.py; notebook |
| Seguridad | F05 y F22; N07, N10 y N11 | security/; tools/policy.py; config.py |
| Experiencia de usuario | I01; N12 y N13 | frontend/src/ |
| Producto Windows | F26 a F28 | Objetivos G a J del plan de productización |

Los RF 01 a RF 13 y RF 15 de la arquitectura previa se distribuyen entre las funciones anteriores. El antiguo RF 14 de traza se limita aquí a consulta de estado y eventos realmente disponibles. El antiguo RF 16 de cancelación activa queda fuera del compromiso mientras la API continúe respondiendo 501. El bloqueo total de red, el proceso separado y la observabilidad completa de aquella arquitectura son objetivos futuros, no garantías atribuidas al runtime actual.

## Apéndice D Pendientes y control de cambios

Antes del cierre de producto deben resolverse: evidencia de proveedores y routing reales, voz y micrófono reales, requisitos de hardware de referencia, configuración instalada, launcher, paquete e instalador, y funcionamiento en entorno limpio. Las credenciales y cualquier límite monetario requieren intervención expresa del usuario; nunca se solicitan secretos pegados en el chat.

Un cambio de requisito deberá indicar identificador afectado, motivo, impacto, responsable de decisión y nueva versión documental. Los cambios de estado I, P u O deberán apoyarse en implementación y evidencia correspondiente. No se atribuyen aprobaciones, firmas, presupuestos o fechas de entrega que no se hayan acordado.